CSE 242 — Assignment 3: SVMs, K-means, and K-NN

Learning goals

- Identify support vectors and derive a maximum-margin linear boundary.
- Calculate and interpret a geometric margin.
- Implement K-means assignment and centroid-update steps.
- Observe how the number of clusters changes a clustering result.
- Use K-NN voting to repair noisy training labels and measure its effect on an SVM.

Question 1: Hard-margin linear SVM — 30 points

Use the following two-feature training set:

Point ID	x1	x2	y
1	1	1	+1
2	1	2	+1
3	2	1	+1
4	0	0	-1
5	1	0	-1
6	0	1	-1

(a) Plot and support vectors — 10 points. Plot the data by hand or with Python and identify all support vectors by eye. Explain your selection. The plotting helper in `assignment3_template.py` labels the points for you.

> The support vectors are points $1, 5, 6$ because they touch the edges of the widest separating strip. Points 5 and 6 lie on $x_1 + x_2 = 1$, while point 1 lies on the parallel line $x_1 + x_2 = 2$. The decision boundary lies halfway between these lines; the other training points lie farther away.

(b) Boundary and margin — 20 points. Using the support vectors, derive the maximum-margin separating line **w1 x1 + w2 x2 + b = 0** (10 points), then calculate the geometric margin (10 points). Include the intercept b. Assume a linear, hard-margin SVM with no slack variables. Report the margin as the distance from the decision boundary to the closest training points; if you also report the full width between supporting boundaries, label it separately.

Complete `identify_support_vectors()` and `derive_svm_separator()`. Show your derivation in the written report; fitting a library SVM alone does not supply the requested mathematical work.

> With the standard hard-margin SVM scaling, support vectors satisfy $w^\top x + b = y$. Thus, points 1, 5, and 6 give the equations $w_1 + w_2 + b = 1$, $w_1 + b = -1$, and $w_2 + b = -1$.
>
> Subtracting the second equation from the first gives $w_2 = 2$, and subtracting the third from the first gives $w_1 = 2$. Substituting back gives $b = -3$. Therefore, the separating line is $2x_1 + 2x_2 - 3 = 0$.
>
> To verify that this separator maximizes the margin, consider the constraints $w_1 + w_2 + b \geq 1$, $w_1 + b \leq -1$, and $w_2 + b \leq -1$. These imply $w_2 \geq 2$ and $w_1 \geq 2$, so any feasible solution must have $\|w\|^2 \geq 8$. Our solution attains this lower bound. Its functional margins $y_i(w^\top x_i + b)$ for points 1 through 6 are $(1, 3, 3, 3, 1, 1)$, so all training constraints are satisfied. It therefore minimizes $\|w\|^2$ and maximizes the geometric margin.
>
> The geometric margin is $\gamma = 1/\|w\| = 1/\sqrt{2^2 + 2^2} = 1/(2\sqrt{2}) \approx 0.3536$. This is the perpendicular distance from the decision boundary to the closest training points. The full width between the supporting lines is $2\gamma = 1/\sqrt{2} \approx 0.7071$.

Question 2: K-means — 30 points

(a) Implement the update steps — 15 points. In `kmeans_template.py`, complete `update_assignments()` and `update_centroids()` from scratch. NumPy operations are allowed; do not substitute a library K-means implementation. The outer loop is provided. Document how you resolve distance ties and handle empty clusters.

> Distance ties are resolved by assigning each point to the lowest-index centroid, while empty clusters retain their previous centroid.

(b) Run and plot — 15 points. Generate the provided toy dataset:

```
make_blobs(n_samples=4000, centers=4, cluster_std=0.60, random_state=0)
```

Run your implementation for **k = 2, 3, 4, 6, 10**. Save a plot of the cluster assignments and final centroids for each value. In a short paragraph, explain what changes as k increases and how the results relate to the four generating groups.

The supplied initializer samples data rows with replacement, matching the source notebook, and accepts a seeded random generator. Duplicate initial centroids are possible. Do not use the true blob labels during clustering; they may be used in an optional reference plot.

> When we originally start with the $k = 4$, there is a clear division amongst the groups and the k-means method is able to properly capture this information. However, once we get to $k = 2$ or $k = 3$, it merges the groups and is able to, for the most part, keep the original groups together, but fails on a few points for the $k = 2$ case. The $k = 6$ and $k = 10$ cases keep the original groups together, but with some arbitrary divisions.

Question 3: Noisy-label SVM and K-NN repair — 40 points

Work in `noisy_classification_template.py`. Helpers for generating clean data and training/evaluating the SVM are provided.

Generate independent training and test sets. Each set contains:

- 1,000 positive examples from a Gaussian with mean **(1, 1)** and covariance **I**.
- 1,000 negative examples from a Gaussian with mean **(-1, -1)** and covariance **3I**.

Keep the test labels clean throughout.

(a) Noisy-label baseline — 20 points. Complete `corrupt_training_labels()` and the baseline portion of `run_comparison()`. Independently flip each original positive training label with probability **0.35**, and each original negative training label with probability **0.20**. Apply the probabilities to the original labels so that an example cannot be flipped twice. Train an RBF SVM with **C = 1** and **gamma = 0.01**.

Keep the generated training and test sets fixed, repeat label corruption and training **20 times**, and report the mean accuracy on the clean test set. Use a seed for reproducibility.

> Mean accuracy: 0.8280
> Random Seed: 0

(b) K-NN label repair — 20 points. Complete `repair_labels_knn()` and the comparison portion of `run_comparison()`. For each training point:

1. Find its **five nearest other training points** using Euclidean distance.
2. Replace its label with the majority of those neighbors' noisy labels.
3. Compute every repaired label from the same original noisy-label array, rather than modifying labels sequentially.

A neighbor-search library is allowed. Five binary votes cannot tie; use a consistent rule if distances tie when selecting neighbors. Train a fresh SVM with the same parameters on the repaired labels. For each trial, use the same features and corrupted labels as the baseline.

Save a CSV with one row per trial and columns `trial`, `baseline_accuracy`, and `knn_accuracy`. Report both mean accuracies and their difference. Briefly explain whether K-

NN repair helped and why it might help or hurt. Improvement is not guaranteed. No parameter sweep or additional clustering-based repair experiment is required.

The repair method may use only training features and noisy training labels. Do not pass it clean training labels or test labels. K-NN is a preprocessing step here; the final classifier is an SVM.

> Mean baseline accuracy: 0.8280 (82.80%).
> Mean accuracy after K-NN label repair: 0.8644 (86.44%).
> Mean change: +0.0364, or **3.64 percentage points**.
>
> In this run, K-NN label repair increased mean test accuracy from 82.80% to 86.44%. It can help smooth isolated mislabeled points, but it may hurt when nearby points belong to different classes or when noise affects local neighborhoods.
>
> For equal distances, the lower original row index is selected first.

Setup and commands

Use Python **3.9 or newer**. From this folder:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Use a clean environment to avoid conflicts with existing NumPy/Matplotlib installations.

Generate the Question 1 data plot:

```
python assignment3_template.py --task plot
```

This saves `svm_data.png` beside the script. After completing the TODOs, run:

```
python assignment3_template.py --task svm
python kmeans_template.py --seed 0 --max-iterations 100
python noisy_classification_template.py --repeats 20 --seed 0
```

All three scripts support `--help`. Solution modes intentionally raise `NotImplementedError` until completed. The default output destinations for Questions 2 and 3 are `results/kmeans/` and `results/noise_comparison.csv`; the plotting/saving steps for these questions are student TODOs.

Deliverables

- The three completed Python files.
- A PDF write-up with the SVM derivation and margin, K-means plots and discussion, and the two-method accuracy comparison.
- The experiment CSV.
- Acknowledgment of outside help, e.g., generative AI transcript.